

Efficient Semifield Convolutions

Peter Adema
14460165

Honours Thesis Extension
Credits: 6 EC

Bachelor *Kunstmatige Intelligentie*



University of Amsterdam
Faculty of Science
Science Park 900
1098 XH Amsterdam

Supervisor
Dr. ir. R. van den Boomgaard

Informatics Institute
Faculty of Science
University of Amsterdam
Science Park 900
1098 XH Amsterdam

Semester 2, 2024-2025

Abstract **TODO**

Acknowledgements **TODO**

Contents

1	Introduction TODO	3
1.1	Related work TODO	4
2	Convolutional derivatives	5
2.1	PyTorch reduction operators	5
2.2	Efficient derivatives for custom kernels	7
2.3	Selection-based semifield addition	8
2.4	Invertible semifield addition	8
3	CUDA semifield convolutions	9
3.1	Compute Unified Device Architecture	9
3.2	Parallel Thread Execution and SASS	10
3.3	Numba-CUDA Just-In-Time compilation	10
3.4	PyTorch Integration	11
3.5	Dispatch overhead	11
3.6	Atomics	12
3.7	Shared Memory	12
4	Results	13
4.1	Verification	13
4.2	Timing setup TODO	14
4.3	Performance TODO	15
5	Conclusions TODO	16
5.1	Findings TODO	16
5.2	Discussion TODO	16
5.3	Contributions TODO	17
5.4	Further research TODO	17
5.5	Reproducibility TODO	17
5.6	Ethics Maybe?	17
6	Appendix	19

Chapter 1

Introduction **TODO**

1.1 Related work **TODO**

Chapter 2

Convolutional derivatives

To perform semifield convolutions within the context of a deep-learning application, we must ensure that we can take the gradient with respect to our inputs in an efficient manner for all operations we seek to perform. The primary challenge in this case is not necessarily the gradient for $\otimes$, as this is a scalar function applied once within each path in the computational graph. Instead, the challenge lies in calculating the derivative of the semifield summation $\oplus$, as all terms within this reduction could influence the gradient of all others. While it would be possible to simply perform backpropagation through every instance of $\oplus$ separately, this would be very computationally expensive and likely numerically unstable. As such, we would like to define semifield operators for cases where a more efficient method for calculating the derivative of $\oplus$ can be found.

We have identified three cases in which this is possible and will discuss them in order of increasing implementation complexity. First, we will examine the possibility of using operators provided by PyTorch to implement semifield convolutions. Afterwards, we will show the two common cases for which a reduction has a simple analytical derivative, namely for those operators $\oplus$ which act like max and those which are invertible.

2.1 PyTorch reduction operators

The most straightforward method for implementing semifield convolutions for PyTorch models is clearly to use reduction operators provided by PyTorch itself. These operators have their derivatives provided by PyTorch and are integrated into the PyTorch automatic differentiation system such that writing a backward pass is not required.

The challenge when using PyTorch operators lies not in their derivatives but in their correct application. The axes we wish to reduce over are not the image axes present in the input itself but instead the window axes of every sliding window we extract during the convolution.

A built-in method for accomplishing this is `nn.Unfold`, an operator which examines sliding windows from the input and copies each separately to a new buffer, allowing subsequent operators to work with the window axes of the new buffer. This copying is rather suboptimal, however: if we take a relatively small 32×32 image and extract relatively small 5×5 windows from it, then the result is more than 19 times the size of the image: even ignoring the cost of copying, larger inputs or kernel sizes would quickly lead to memory issues.

As such, we would like to perform a similar operation — transforming the input image such that we have access to sliding window axes to reduce over — but without performing a copy. This transformation is possible by creating an adjusted view of the input image: some other libraries include this feature by default, but it is not exceedingly complex to replicate.

The main idea of using such a view is to adjust the strides (step sizes within the underlying data array) such that, when moving to the next window, the view takes a step equal to the convolutional STRIDE instead of the full size of the window. In Fig. 2.1, we can see how a view into a 1D-array with STRIDE = 1 and size-4 kernels causes subsequent windows to overlap:

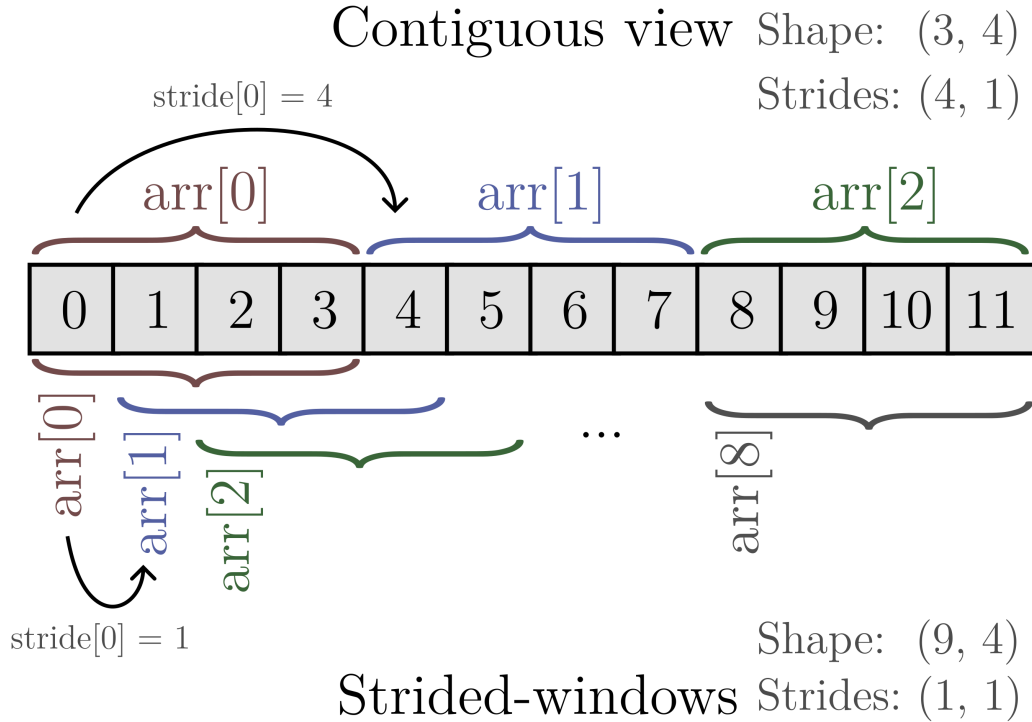


Figure 2.1: Two views onto the same underlying data: at the top, a contiguous view (the typical view for PyTorch arrays), and at the bottom, a strided-windows view (allowing for extracting windows without copying).

By applying a broadcasting version of $\otimes$ to the strided view and the convolutional kernels, we can obtain the intermediaries to be reduced. Subsequently, we can apply the reduction operator along the window axes (axis 1 in Fig. 2.1) to efficiently calculate $\oplus$ for all cases where $\oplus$ and $\otimes$ can be expressed as one or more PyTorch operators.

However, this approach still has a major downside in terms of memory usage and cache locality: there is currently no method for fusing the $\otimes$ and $\oplus$ operations in PyTorch, and the intermediary results of $\otimes$ must therefore be manifested in their entirety. As a result, this method still suffers from greatly increased memory use and also accesses memory inefficiently. Since all results of $\otimes$ are stored before $\oplus$ begins, the initial results of $\otimes$ are usually flushed from the cache by the beginning of $\oplus$, causing cache-misses and further slowdown.

In order to mitigate this, we must devise custom, fused kernels that combine $\otimes$ and $\oplus$ into a single operation. Such kernels, however, also require calculating the derivative rules manually. As such, the following two sections will discuss two methods for deriving the reduction $\oplus$ when $\oplus$ is a selection or is invertible.

2.2 Efficient derivatives for custom kernels

If we consider the operations that take place within the convolutional operation, then the computational graph for any one point in the result is an unbalanced binary tree, with a sequence of $\oplus$ operations along the spine constituting the reduction. If we would truly wish to derive this computational graph as-is, then the gradients propagated to the bottom of the tree may suffer from numerical instability due to the overly high number of operations to backpropagate through. Instead, we would prefer cases in which we could skip deriving through the reduction and calculate the derivative of the result with respect to any term in the reduction directly. This difference is illustrated in Fig. 2.2, where the right computational graph shows these direct derivatives:

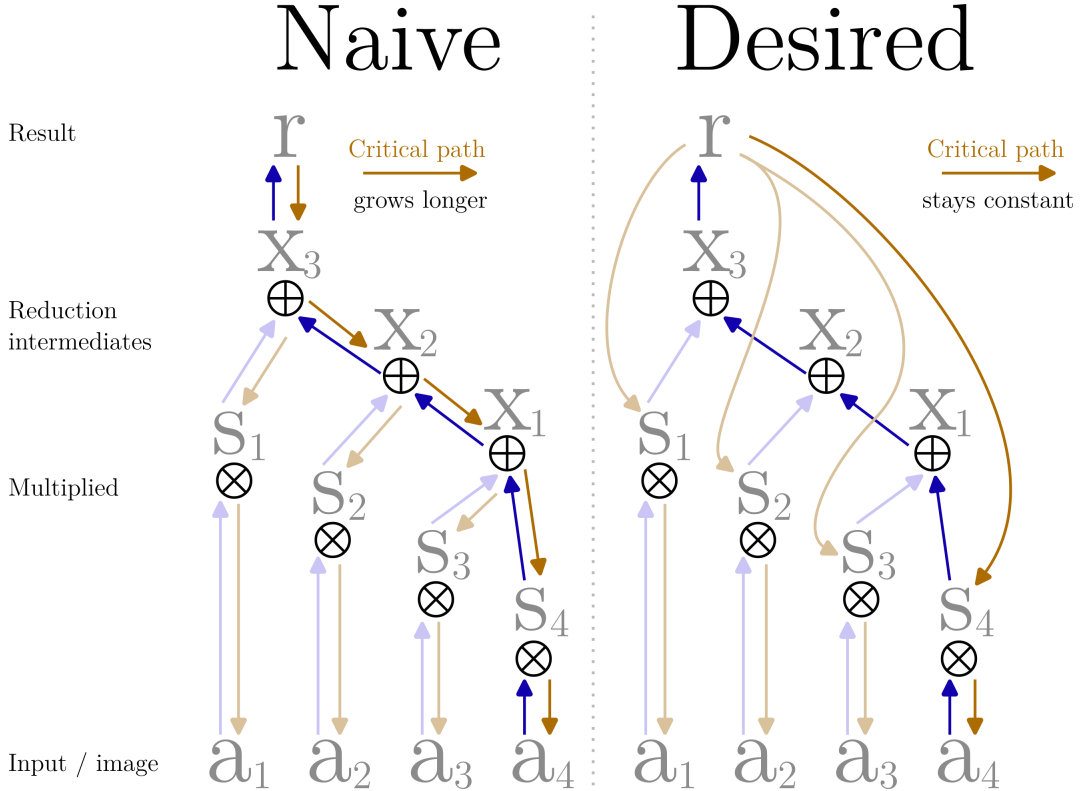


Figure 2.2: Two examples of computational graphs for a semifield convolution at a single point, with the left graph showing a naive implementation of the backwards graph, and the right graph showing a more efficient, desired version. The right graph is only relevant if $\frac{\delta r}{\delta s_n}$ is easily computable for any n .

Unfortunately, $\frac{\delta r}{\delta s_n}$ is not easily computable for any n in all semifields. In order to attain the desired behaviour for the backward pass, we must therefore place additional constraints on the operator $\oplus$; it must either:

- Act as a selection — similar to max and min — where r is always takes on the value of exactly one of the inputs s . See Sec. 2.3 for details.
- Be invertible (as well as commutative and associative), such that if $r = x \oplus s$, then x can be found given r and s . See Sec. 2.4 for details.

These two cases are the only cases for which a custom kernel is implemented in the package written alongside this report, corresponding with `SelectSemifield` and `SubtractSemifield` respectively.

2.3 Selection-based semifield addition

When $\oplus$ acts as a selection operator, its result is always equal to one of its two operands. The typical examples of selection operators are max and min, which always select the higher and lower operands, respectively. However, it is conceivable that one may wish to perform a maximum-like operation using an alternative ordering, hence this generalisation.

The advantage of selection-based semifield addition is that the result of the reduction $\bigoplus$ must also be equal to one of its inputs (an s from Fig. 2.2) while having no relation to any of the other inputs. As such, we can see that the derivative $\frac{\delta r}{\delta s_n}$ is equal to 1 for the s_c , which r takes its value from, and 0 for all s_i where $i \neq c$. As such, letting the backpropagation 'skip over' the reduction would only require knowing the value of c (the index of the selected s_c).

While it is possible to recompute this c using a second forward pass during backpropagation, we can — using a small amount of auxiliary memory — memoise this index c during the initial forward pass. By storing this c (also known as the provenance index), the backward pass only needs to update the gradient of the value at index c and can ignore all other terms in the reduction. The backward pass, therefore, only requires recomputing s_c , not the other s 's.

2.4 Invertible semifield addition

For all other operators that we would wish to use as $\oplus$, simple memoisation of the result will not suffice. There is, however, still a class of operators which leads to relatively simple backward graphs. If we examine again the naive backwards graph from Fig. 2.2, we can see that, although the path of the gradient is relatively long for most s_i , the path for s_1 is relatively short. The last value added in the reduction only requires the derivative of one instance of $\oplus$, greatly simplifying the calculation. However, for any specific ordering of the values $s_1 \dots s_n$, there is only one instance of this 'easy case'. To solve this, we could require an associative and commutative $\oplus$, and write for any s_i [1]:

$$r = s_1 \oplus s_2 \oplus \dots s_{i-1} \oplus \mathbf{s_i} \oplus s_{i+1} \oplus \dots s_{n-1} \oplus s_n \quad (2.1)$$

$$= (s_1 \oplus s_2 \oplus \dots s_{i-1}) \oplus \mathbf{s_i} \oplus (s_{i+1} \oplus \dots s_{n-1} \oplus s_n) \quad (2.2)$$

$$= \mathbf{s_i} \oplus (s_1 \oplus s_2 \oplus \dots s_{i-1} \oplus s_{i+1} \oplus \dots s_{n-1} \oplus s_n) \quad (2.3)$$

$$= \mathbf{s_i} \oplus x_{\setminus i} \quad (2.4)$$

In other words, we can rewrite the reduction for any s_i into this 'easy case', where the result is equal to s_i combined with the accumulation of all other values. The challenge then becomes finding this $x_{\setminus i}$: simply performing the reduction for every i separately would be highly inefficient, so a better solution would be required. If $\oplus$ is invertible, then the solution is straightforward: by Eq. 2.4 we know that $r = s_i \oplus x_{\setminus i}$, so if we recalculate s_i and memoise r (from the forward pass), then $x_{\setminus i}$ can be found using the inverse of $\oplus$: $x_{\setminus i} = r \overset{inv}{\oplus} s_i$. Given both s_i and $x_{\setminus i}$, we can then easily calculate the gradient of Eq. 2.4.

In practice, the restriction of invertibility is not overly constraining. All fields are necessarily invertible (we can calculate $x_{\setminus i}$ by taking the element which is the additive inverse of s_i and seeing that $x_{\setminus i} = r \oplus s_i^{inv}$). Furthermore, even many semifields which are not themselves fields have inverses, such as the log semifields $L_{\pm\mu}$ and the root semifields R_p . In these cases, by replacing the $+$ within the semifield addition with $-$, we can obtain the inverse.

Chapter 3

CUDA semifield convolutions

Armed with an understanding of the types of semifield convolutions where a gradient can be calculated in a reasonably efficient manner, we now turn to the task of efficiently implementing these operations as programs that can run on a modern NVIDIA GPU as CUDA kernels. First, we will discuss how programs that can run on an NVIDIA GPU are represented, as well as how such programs can be written in Python using Numba-CUDA. Subsequently, we will show how the overhead involved in dispatching GPU programs ('kernels') can be reduced, and we will discuss some features of CUDA used to improve performance.

3.1 Compute Unified Device Architecture

Commonly referred to as CUDA, this programming model allows for describing operations in an inherently parallel fashion. Typically, CUDA programming involves splitting the parallelisable task into its smallest parts and spawning a thread for every individual task (for machine learning, often one thread per position in the target array). Threads are then hierarchically grouped, with 32 threads forming a 'warp' and a variable¹ number of warps forming a 'block'².

Within a warp, threads can communicate using special instructions that transfer data between registers of a warp. For more structured cooperation, threads within a block can access 'shared memory', which is most similar to a fast version of heap-allocated memory for CPU threads. Blocks can further be arranged into 'clusters' if more warps are required to work on the problem than fit within one block, but this is rarely used (also not for this report).

In terms of actual execution, an operation (such as a semifield convolution) consists of blocks arranged into a 'grid'. When a grid is executed, some of its blocks are assigned to a Streaming Multiprocessor (SM), where all their threads are run from start to finish. In an SM, warps are issued instructions, with all threads in a warp receiving (and potentially skipping) the same instructions. In a sense, warp instructions are highly optimised and flexible versions of predicated SIMD instructions³. However, besides using this data parallelism to accelerate computation, CUDA devices can also leverage concurrency (akin to CPU hyperthreading) to hide memory latencies. Every computational unit ('scheduler', 4 per SM) can house several (up to 16) warps and can retain optimal performance so long as one of the warps is always eligible for execution.

¹On most hardware, a block may have between 1 and 32 warps (i.e. 32 to 1024 threads).

²Also called a 'thread block', or a 'cooperative thread array' in PTX documentation.

³The model of warps is also referred to as Single Instruction Multiple Threads (SIMT)

3.2 Parallel Thread Execution and SASS

Besides describing a device architecture, the term 'CUDA' is often used to refer to the CUDA C extension to the C programming language, which allows users to write programs that run on CUDA devices in a dialect of C or C++. However, just like C does not run directly on a CPU, CUDA C itself cannot run on CUDA devices: it must first be compiled to machine instructions.

The assembly format of choice for CUDA devices is Streaming Assembler (SASS), which compiles into executable bytecode. Most compiler frontends that target CUDA devices do not output SASS directly, however. Instead, a more cross-compatible assembly format named Parallel Thread Execution (PTX) is used, and an NVIDIA transpiler (`ptxas`) is used to convert PTX into SASS at runtime. The idea behind this construction is similar to that of VM-based languages, where developers can write programs in a high-level language (such as CUDA C), compile to a backwards- and forwards-compatible format for distribution and have their code run on a wide range of devices.

3.3 Numba-CUDA Just-In-Time compilation

CUDA C is not the only frontend for PTX; the Numba-CUDA package provides an alternative based on Python. By providing the same intrinsics that CUDA C offers, Numba-CUDA enables writing programs for CUDA devices in relatively standard Python. Unlike CUDA C, applications using Numba-CUDA are not distributed with their GPU code compiled to PTX but instead leave this compilation step as a runtime task as well (in addition to the task of transpiling the PTX to SASS and assembling the SASS).

Just-In-Time (JIT) compilation refers to the above technique of delaying compilation to be a one-off runtime cost instead of being a preparatory step. JIT compilers and applications have the advantage of being able to specialise at runtime — certain aspects of the input can be assumed to be constant (such as the shape of an image or the dilation of a convolution), providing more room for optimising compilers to improve performance. The primary downside of JIT compilers is usually taken to be the increased start-up times associated with runtime compilation. However, these costs are negligible in machine-learning contexts, where training can consist of thousands or millions of iterations.

Due to the reduced complexity of programming in Python, as well as the increased flexibility of runtime specialisation, it was chosen to write the packages for this report using Numba-CUDA as the frontend instead of CUDA C.

The full compilation pipeline is visualised for clarity in Fig. 3.1:

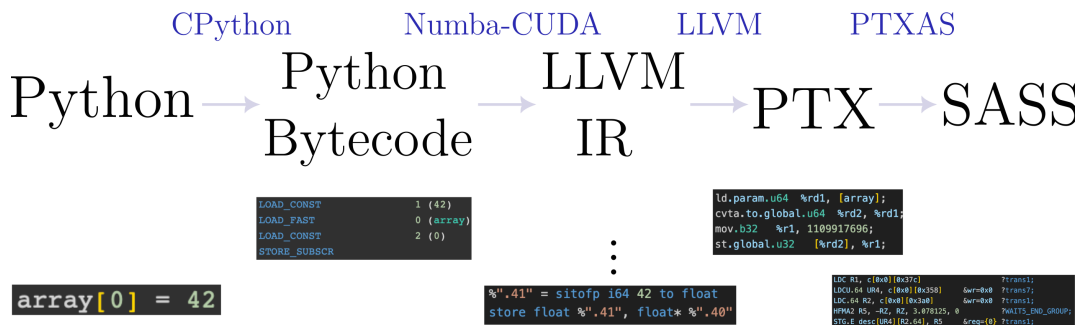


Figure 3.1: Compilation pipeline from Python source code to SASS bytecode.

3.4 PyTorch Integration

Many projects — including this one — performing research in machine-learning use the PyTorch framework to arrange operations into a computational graph for backpropagation or compilation. However, while a sequence of PyTorch built-in operators can typically be traced to form such a graph, this is not the case for custom CUDA operators. In these cases, more care is required in order to use these operators within the PyTorch automatic differentiation system.

Using `torch.library`, custom operators can be defined such that PyTorch tracing systems (`dynamo`) can properly integrate the operators in their graphs. However, writing such code in Python can lead to a large per-call dispatch overhead for these operators; due to various conversions and checks taking place in the process of invoking a custom Python operator, these can be much slower than their built-in counterparts. Furthermore, while Numba-CUDA supports the CUDA Array Interface and can interact with PyTorch data, using this Python interface also causes slowdowns. The total additional overhead can cause operators to be significantly slower (by up to an order of magnitude, see Sec. 4.3). As such, resolving this slowdown is also an important factor to consider before attempting to use these operators to train models.

3.5 Dispatch overhead

One approach to resolving this slowdown is to rewrite the operators as compiled (C++) extensions to the PyTorch framework. The primary downside of this approach would be that it would require source code for the operators to be available for compilation, but the generic nature of semifield convolutions are very challenging to express for ahead-of-time compilation. This can be resolved by instead making the C++ extensions also JIT-compiled; by using Numba-CUDA to compile dynamic Python code, the resulting PTX can be embedded in a C++ function without much knowledge of its workings. This construct, where Numba-CUDA is used to JIT-compile PyTorch extensions, is implemented by the `pytorch-numba-extension-jit` package written for this report, and can eliminate much of the Python-induced overhead from custom operators in PyTorch computation graphs.

However, a better method is available for the purposes of training models. In general, the nature of any overhead within the training process originates from the CPU being unable to start GPU programs faster than the GPU is able to complete them. While this cannot be avoided in cases where the programs to be executed are different between runs, training models in machine-learning typically involves running a fairly static forward- and backward-pass for many iterations. In such cases, an alternative approach named CUDAGraphs can allow a CUDA GPU to record a series of operations, and then repeat them with only a single invocation (like a 'macro' from productivity software). Using CUDAGraphs, dispatch overhead from custom operators can be eliminated entirely, without even resorting to JIT-compilation of C++ extensions.

Since the main report is primarily concerned with training and evaluating various machine-learning models, CUDAGraphs is used without additional C++ compilation to reduce dispatch overhead. For other cases, where the process of CUDAGraphs tracing is too costly, C++ extension compilation may still be an effective method.

3.6 Atomics

Within a CUDA GPU, there are many features designed to accelerate a variety of computational tasks. Most of these features cannot effectively be applied for the purpose of semifield convolutions, save the feature of atomic additions. While the forward pass of both a select-based semifield and a invertible semifield can be written with standard Python constructs, the backward pass of both requires aggregating the gradients for various positions in the image and kernel. It is possible to do this aggregation using a convolution-like operation in reverse, i.e. for every position in the input, sum over the gradients with respect to every output position that could have accessed it. However, this is suboptimal in the case of select-based semifields, as every output position only corresponds with at most one input position. Atomic additions allow threads to cooperate on a summation: multiple threads can concurrently perform an addition on a memory address (e.g. a part of the gradient for the input), and the hardware will guarantee that all summations are performed in some order. This is important, as several threads attempting to add a value to a specific address might otherwise overwrite each-other's work. By leveraging atomics, the backward pass for select-based semifields can be performed efficiently, in cases even faster than a built-in PyTorch max-pooling (see [TODO graph](#)).

3.7 Shared Memory

Besides atomic operations, another feature which is often used for CUDA programs is 'shared memory', whereby threads in a block can access a shared block of memory to cooperate on a task. Often, shared memory is used to store intermediate results or to cache input values for more efficient access.

For semifield convolutions, however, shared memory is challenging to apply effectively. The first use-case — caching intermediates — does not especially play a role within semifield convolutions, as there are no clear intermediate steps within the calculation. Breaking up the window and assigning multiple threads per output position is an option, but decreased performance during testing. As such, the only possible use-case for shared memory would be in caching inputs.

The argument in favour of this use case is that accessing uncached, global memory is slow if all memory accesses are not clustered together (within 128 bytes), while a slightly laxer set of limitations apply for shared memory accesses. As such, storing the input image or kernel might allow for more efficient access to it. While this seems reasonable, it is not the case in practice; it is possible to write the convolution in a manner where memory access is mostly linear and predictable, whereafter the automatic caching system (akin to CPU L1/L2 cache) can correctly keep all relevant data in the cache. As such, while many variations on shared memory caching were tried — from copying the entire relevant parts of the image/kernel to maintaining a local sliding window of the image — but none could outperform the basic version which relies on automatic caching, and most performed significantly worse.

Considering the above, it was chosen to not use any shared memory or other synchronisation tools within the implementations of the semifield convolutions, using only atomic addition as non-standard feature.

Chapter 4

Results

Created using the derivative schemas from Chapter 2 and the technical tools described in Chapter 3, the package `pytorch-nd-semiconv` implements the three types of semifield convolutions discussed and provides integrations for use with the PyTorch machine-learning framework. The operators work with arbitrary dimensionality (e.g. 1D signals / 2D images / 3D imaging data). Using this package, some verification experiments were conducted and timings were done to examine correctness and performance of the resulting code.

4.1 Verification

In order to ensure that the results for the main report are not misrepresenting the characteristics of semifield convolutions, it was essential to ensure that the operators in `pytorch-nd-semiconv` are correct and abide by expectations.

Verification of these operators was done in three parts. First, a set of simple experiments were done using fixed data, with results and gradients verified to be identical to their manually calculated counterparts. Secondly, theoretical characteristics of morphology were used to verify the correctness of dilations (T_+) and erosions (negated T_-) specifically. Finally, a variety of random inputs were used to verify that the CUDA implementations behaved identically to their PyTorch broadcasting counterparts.

Regarding the fixed data experiments, interested readers may consult the Jupyter Notebook `groups_test.ipynb` to view the results of the operators on some simple data and using convolutional groups.

For the morphological characteristics, it was chosen to focus on verifying the extensivity of closings (that the results of a closing is always $\geq$ the input) as well as the dual anti-extensivity of openings ($\dots \leq$). Using both random and fixed data, these characteristics were verified to hold in `plot_adjunction.ipynb`.

Finally, an extensive test covering 100 random configurations for input dimensionality, input and kernel size, stride, dilation and padding was run, where the results and gradients of the PyTorch broadcasting implementation were verified to be identical to the corresponding implementation using CUDA code. `ops_equivalent.py` showed that the implementations were equivalent in nearly all¹ cases, strengthening the case for the correctness of the operators.

¹The derivative of semifield multiplication does not always consider the absorbing nature of the semifield zero: $\frac{\delta(x+\infty)}{\delta x} = 1$ in PyTorch, even when changing x would not change the result of $x+\infty$. As such, when the kernel contains a semifield zero (due to it being set as such) or the input does (due to e.g. padding), the gradient can differ between implementations.

4.2 Timing setup **TODO**

4.3 Performance **TODO**

Chapter 5

Conclusions **TODO**

5.1 Findings **TODO**

5.2 Discussion **TODO**

- 5.3 Contributions **TODO**
- 5.4 Further research **TODO**
- 5.5 Reproducibility **TODO**
- 5.6 Ethics **Maybe?**

.....

Bibliography

- [1] A. Paszke, M. J. Johnson, R. Frostig, and D. Maclaurin, “Parallelism-preserving automatic differentiation for second-order array languages,” in *Proceedings of the 9th ACM SIGPLAN International Workshop on Functional High-Performance and Numerical Computing*, 2021, pp. 13–23.

Chapter 6

Appendix